🚀 Step 1.5: Basic Recursion Master Revision Guide

A complete, interview-ready reference for Striver's Basic Recursion module in Python. Contains problem-by-problem patterns, clean implementations, line-by-line code breakdowns, and time/space complexity analysis.

📌 Summary Table

#	Problem File	LeetCode #	Core Pattern	Time	Space
1	`1.Print 1 to N using Recursion.py`	-	Forward Head Recursion	$O(N)$	$O(N)$
2	`2.Print name N times using recursion.py`	-	Counter Recursion	$O(N)$	$O(N)$
3	`3.print n to 1 time using recursion.py`	-	Decrement Head Recursion	$O(N)$	$O(N)$
4	`4.print 1 o n using backtracking.py`	-	Stack Unwinding (Backtracking)	$O(N)$	$O(N)$
5	`5. sum of first n numbers using parameterised.py`	-	Parameterized Accumulation	$O(N)$	$O(N)$
6	`6.sum of first n numbers using functional recursion.py`	-	Functional Return Unwinding	$O(N)$	$O(N)$
7	`7.sum of n numbers using factorial.py`	-	Multiplicative Functional Unwinding	$O(N)$	$O(N)$
8	`Reverse an array using recursion.py`	-	Single-Pointer Symmetric Swap	$O(N)$	$O(N)$
9	`Check if String is Palindrome using Recursion.py`	#125	Single-Pointer Symmetric Compare	$O(N)$	$O(N)$
10	`Fibonacci Number.py`	#509	Tree Recursion (Multiple Calls)	$O(2^N)$	$O(N)$

🧠 Comprehensive Problem Breakdown

1. Print 1 to N using Recursion

- **File:** `1.Print 1 to N using Recursion.py`
- **Pattern:** Forward Head Recursion
- **Intuition:** Perform the print operation *before* making the recursive call. Execution happens on the way **down** the call stack ($1 \rightarrow N$).

```python
class Solution:

    def print_1_to_n(self, i, n):
        # Line 1: Base Case - Terminate when counter exceeds target 'n'
        if i > n:
            return

        # Line 2: Work - Print current index on the way down
        print(i)

        # Line 3: Recurse - Advance counter by 1
        self.print_1_to_n(i + 1, n)
```

- **Line 1 (`if i > n: return`):** Stopping condition that prevents stack overflow once `i` goes beyond `n`.
- **Line 2 (`print(i)`):** Executes before the function calls itself, printing values in increasing order ($1 \to N$).
- **Line 3 (`self.print_1_to_n(i + 1, n)`):** Pushes the next frame onto the stack with `i + 1`.
- **Complexity:** Time: $O(N)$, Space: $O(N)$ (auxiliary call stack).

---

2. Print Name N Times

- **File:** `2.Print name N times using recursion.py`
- **Pattern:** Counter Recursion
- **Intuition:** Replicates a standard loop using a counter parameter `i`. Prints on the way down until `i > n`.

```python
class Solution:

    def print_name(self, i, n):
        # Line 1: Base Case - Stop when counter exceeds 'n'
        if i > n:
            return

        # Line 2: Work - Print target string
        print("Shahbaaz")

        # Line 3: Recurse - Move counter forward
        self.print_name(i + 1, n)
```

- **Line 1 (`if i > n: return`):** Halts the function after N executions.
- **Line 2 (`print("Shahbaaz")`):** Performs the desired printing task.
- **Line 3 (`self.print_name(i + 1, n)`):** Moves the state closer to the base case.
- **Complexity:** Time: $O(N)$, Space: $O(N)$.

---

3. Print N to 1 using Recursion

- **File:** `3.print n to 1 time using recursion.py`
- **Pattern:** Decrement Head Recursion
- **Intuition:** Counter starts at N and decrements towards 1. Output is generated on the way down.

```python
class Solution:

    def print_n_to_1(self, i, n):
        # Line 1: Base Case - Stop when counter drops below 1
        if i < 1:
            return

        # Line 2: Work - Print current decreasing index
        print(i)

        # Line 3: Recurse - Decrement counter
        self.print_n_to_1(i - 1, n)
```

- **Line 1 (`if i < 1: return`):** Triggers termination when `i` reaches 0.
- **Line 2 (`print(i)`):** Prints values in descending order ($N \to 1$).
- **Line 3 (`self.print_n_to_1(i - 1, n)`):** Decrements `i` towards 0.
- **Complexity:** Time: $O(N)$, Space: $O(N)$.

---

4. Print 1 to N using Backtracking

- **File:** `4.print 1 o n using backtracking.py`
- **Pattern:** Stack Unwinding (Backtracking)
- **Intuition:** Recurse deep to 1 first without printing, then execute `print(i)` **after** the recursive call during stack unwinding.

```python
class Solution:

    def print_1_to_n_bt(self, i, n):
        # Line 1: Base Case - Reached bottom of call stack
        if i < 1:
            return

        # Line 2: Recurse - Push deeper frames BEFORE printing
        self.print_1_to_n_bt(i - 1, n)

        # Line 3: Work - Executes during stack unwinding (bottom-up)
        print(i)
```

- **Line 1 (`if i < 1: return`):** Hits base case at `i = 0`.
- **Line 2 (`self.print_1_to_n_bt(i - 1, n)`):** Postpones printing and pushes calls onto the stack until `i = 0`.

- **Line 3 (`print(i)`):** Executes bottom-up as calls return, printing in increasing order ($1 \to N$).
- **Complexity:** Time: $O(N)$, Space: $O(N)$.

---

5. Sum of First N Numbers (Parameterized)

- **File:** `5. sum of first n numbers using parameterised.py`
- **Pattern:** Parameterized Accumulation
- **Intuition:** Pass running accumulator down as an argument (`sum_val + i`). The final answer is ready at the base case.

```
class Solution:

    def sum_parameterized(self, i, sum_val):
        # Line 1: Base Case - Print accumulated sum at the bottom
        if i < 1:
            print(sum_val)
            return

        # Line 2: Recurse - Pass decremented 'i' and updated running total
        self.sum_parameterized(i - 1, sum_val + i)
```

- **Line 1 (`if i < 1:`):** Triggers when all numbers are added. `sum_val` holds the complete answer.
- **Line 2 (`self.sum_parameterized(i - 1, sum_val + i)`):** Passes accumulated total into the next frame.
- **Complexity:** Time: $O(N)$, Space: $O(N)$.

---

6. Sum of First N Numbers (Functional)

- **File:** `6.sum of first n numbers using functional recursion.py`
- **Pattern:** Functional Return Unwinding
- **Intuition:** Defined mathematically as $f(N) = N + f(N-1)$. The function returns values back up the stack during unwinding.

```
class Solution:

    def sum_functional(self, n):
        # Line 1: Base Case - Additive Identity
        if n == 0:
            return 0

        # Line 2: Recurse - Return 'n' + result of smaller subproblem
        return n + self.sum_functional(n - 1)
```

- **Line 1 (`if n == 0: return 0`):** Returns 0 (additive identity) so adding 0 leaves totals unchanged.

- **Line 2 (`return n + self.sum_functional(n - 1)`):** Unwinds bottom-up, adding n to the subproblem answer.
- **Complexity:** Time: $O(N)$, Space: $O(N)$.

---

7. Factorial of N

- **File:** `7.sum of n numbers using factorial.py`
- **Pattern:** Multiplicative Functional Unwinding
- **Intuition:** Defined as $N! = N \times (N-1)!$. Base case **must return 1** (multiplicative identity) to prevent multiplying by 0.

```python
class Solution:

    def fact(self, n):
        # Line 1: Base Case - Multiplicative Identity (0! = 1, 1! = 1)
        if n == 0 or n == 1:
            return 1

        # Line 2: Recurse - Return 'n' * subproblem result
        return n * self.fact(n - 1)
```

- **Line 1 (`if n == 0 or n == 1: return 1`):** Returning 1 prevents destroying the product chain.
- **Line 2 (`return n * self.fact(n - 1)`):** Multiplies n by the subproblem return value during unwinding.
- **Complexity:** Time: $O(N)$, Space: $O(N)$.

---

8. Reverse an Array using Recursion

- **File:** `Reverse an array using recursion.py`
- **Pattern:** Single-Pointer Symmetric Swap
- **Intuition:** Use a single index `i`. Swap `arr[i]` with `arr[n - 1 - i]` until reaching the middle index (`i >= n // 2`).

```python
class Solution:

    def reverse_array(self, arr, i, n):
        # Line 1: Base Case - Reached or passed middle index
        if i >= n // 2:
            return

        # Line 2: Work - Swap symmetric elements in-place
        arr[i], arr[n - 1 - i] = arr[n - 1 - i], arr[i]

        # Line 3: Recurse - Advance pointer towards middle
        self.reverse_array(arr, i + 1, n)
```

- **Line 1 (`if i >= n // 2: return`):** Stops at the midpoint to prevent double-swapping back to original positions.
- **Line 2 (`arr[i], arr[n - 1 - i] = ...`):** In-place swap using tuple packing.
- **Line 3 (`self.reverse_array(arr, i + 1, n)`):** Moves `i` closer to the center.
- **Complexity:** Time: $O(N)$, Space: $O(N)$.

---

9. Check if String is Palindrome using Recursion

- **File:** `Check if String is Palindrome using Recursion.py`
- **LeetCode:** #125 Valid Palindrome
- **Pattern:** Single-Pointer Symmetric Compare
- **Intuition:** Compare `s[i]` with `s[n - 1 - i]`. Short-circuit with `False` on any mismatch. Return `True` when `i >= n // 2`.

```python
class Solution:

    def is_palindrome(self, s, i=0):
        n = len(s)

        # Line 1: Base Case - Reached middle with all matches intact
        if i >= n // 2:
            return True

        # Line 2: Early Exit - Immediate mismatch detected
        if s[i] != s[n - 1 - i]:
            return False

        # Line 3: Recurse - Check next inner pair
        return self.is_palindrome(s, i + 1)
```

- **Line 1 (`if i >= n // 2: return True`):** All character pairs matched successfully.
- **Line 2 (`if s[i] != s[n - 1 - i]: return False`):** Immediate exit on mismatch.
- **Line 3 (`return self.is_palindrome(s, i + 1)`):** Checks inner adjacent pair.
- **Complexity:** Time: $O(N)$, Space: $O(N)$.

---

10. Fibonacci Number (Multiple Calls)

- **File:** `Fibonacci Number.py`
- **LeetCode:** #509 Fibonacci Number
- **Pattern:** Tree / Branching Recursion
- **Intuition:** Each call branches into two recursive subproblems ($F(N) = F(N-1) + F(N-2)$), generating a binary call tree.

```python
class Solution:

    def fibonacci(self, n):
```

```
        # Line 1: Base Cases - F(0) = 0, F(1) = 1
        if n <= 1:
            return n

        # Line 2: Recurse - Sum of left and right subtrees
        return self.fibonacci(n - 1) + self.fibonacci(n - 2)
```

- **Line 1 (`if n <= 1: return n`):** Returns 0 for $n = 0$ and 1 for $n = 1$.
- **Line 2 (`return self.fibonacci(n - 1) + self.fibonacci(n - 2)`):** Evaluates left branch to base case first, then right branch, summing both on unwinding.
- **Complexity:** Time: $O(2^N)$, Space: $O(N)$.